



CMPE 344 – Database Management and Systems II

Term Project

Flight DBMS

Abdulraheem Ba Wazir – 22209358

Saeed Al Harbi – 22206502

Introduction:

The Flight Management System database has been designed to manage airline operations efficiently and accurately. The system organizes and stores data related to users, flights, flight schedules, bookings, tickets, aircraft, airports, and payments. This database provides a structured and reliable way to handle core airline processes such as flight scheduling, customer bookings, ticket issuance, and revenue tracking.

The system assumes the following key functionalities and processes:

- **User Management:** User information is stored with role-based access control. Roles include administrator, employee, customer, and technical staff, each having specific permissions.
- **Flight Management:** The system maintains detailed information about flights, including origin, destination, and base pricing.
- **Flight Scheduling:** Each flight can have multiple schedules with different dates, times, and aircraft.
- **Booking Management:** Customers can create bookings associated with specific flight schedules.
- **Ticket Management:** Tickets are generated per booking, including seat number and ticket price.
- **Payment Handling:** Payment records track booking payments and their status.

The database ensures data integrity through the use of primary keys, foreign keys, and constraints, allowing accurate data retrieval and analysis through advanced SQL queries.

Database:

ERD:

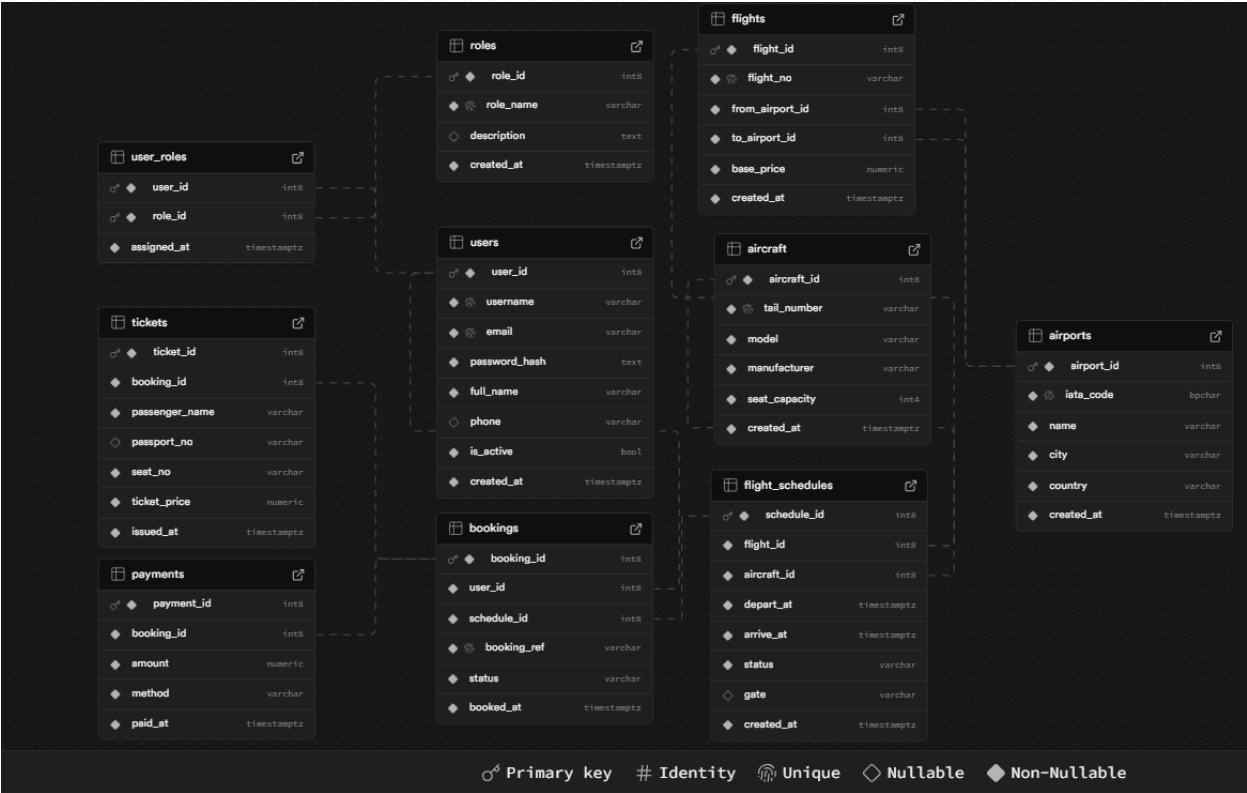
The database was designed using an Entity-Relationship (ER) model to clearly represent entities and their relationships.

The ER Diagram includes the following main entities:

- Users
- Roles
- User_Roles
- Airports
- Aircraft
- Flights
- Flight_Schedules
- Bookings
- Tickets
- Payments

Relationships:

- A user can have multiple roles.
- A flight has multiple schedules.
- A schedule can have many bookings.
- A booking can generate multiple tickets.
- Each ticket belongs to exactly one booking.



Relational Data Model (RDM):

Users (user_id, username, password_hash)

Roles (role_id, role_name)

User_Roles (user_id, role_id)

- FK: user_id → Users(user_id)
- FK: role_id → Roles(role_id)

Airports (airport_id, iata_code, city, country)

Aircraft (aircraft_id, model, capacity)

Flights (flight_id, flight_number, from_airport_id, to_airport_id, base_price)

- FK: from_airport_id → Airports(airport_id)
- FK: to_airport_id → Airports(airport_id)

Flight_Schedules (schedule_id, flight_id, aircraft_id, departure_time, arrival_time)

- FK: flight_id → Flights(flight_id)
- FK: aircraft_id → Aircraft(aircraft_id)

Bookings (booking_id, user_id, schedule_id, booking_date, status)

- FK: user_id → Users(user_id)
- FK: schedule_id → Flight_Schedules(schedule_id)

Tickets (ticket_id, booking_id, seat_number, price)

- FK: booking_id → Bookings(booking_id)

Payments (payment_id, booking_id, amount, payment_status)

- FK: booking_id → Bookings(booking_id)

DDL:

```
sql > ddl.sql
1  drop table if exists payments cascade;
2  drop table if exists tickets cascade;
3  drop table if exists bookings cascade;
4  drop table if exists flight_schedules cascade;
5  drop table if exists flights cascade;
6  drop table if exists aircraft cascade;
7  drop table if exists airports cascade;
8  drop table if exists user_roles cascade;
9  drop table if exists roles cascade;
10 drop table if exists users cascade;
11
12 create table users (
13     user_id      bigserial primary key,
14     username     varchar(50) not null unique,
15     email        varchar(120) not null unique,
16     password_hash text not null,
17     full_name    varchar(120) not null,
18     phone        varchar(30),
19     is_active     boolean not null default true,
20     created_at   timestampz not null default now()
21 );
22
23 create table roles (
24     role_id      bigserial primary key,
25     role_name    varchar(40) not null unique,
26     description  text,
27     created_at   timestampz not null default now()
28 );
29
30 create table user_roles (
31     user_id      bigint not null references users(user_id) on delete cascade,
32     role_id      bigint not null references roles(role_id) on delete cascade,
33     assigned_at  timestampz not null default now(),
34     primary key (user_id, role_id)
35 );
36
37 create table airports (
38     airport_id   bigserial primary key,
39     iata_code    char(3) not null unique,
40     name         varchar(120) not null,
41     city         varchar(80) not null,
42     country      varchar(80) not null,
43     created_at   timestampz not null default now()
44 );
45
46 create table aircraft (
47     aircraft_id  bigserial primary key,
48     tail_number  varchar(20) not null unique,
49     model        varchar(80) not null,
50     manufacturer varchar(80) not null,
51     seat_capacity int not null check (seat_capacity > 0),
52     created_at   timestampz not null default now()
53 );
54
```

```

sql> ddl.sql
55 create table flights (
56     flight_id          bigserial primary key,
57     flight_no          varchar(10) not null unique,
58     from_airport_id    bigint not null references airports(airport_id),
59     to_airport_id      bigint not null references airports(airport_id),
60     base_price          numeric(10,2) not null check (base_price >= 0),
61     created_at         timestampz not null default now(),
62     constraint chk_route_not_same check (from_airport_id <> to_airport_id)
63 );
64
65 create table flight_schedules (
66     schedule_id        bigserial primary key,
67     flight_id          bigint not null references flights(flight_id) on delete cascade,
68     aircraft_id        bigint not null references aircraft(aircraft_id),
69     depart_at          timestampz not null,
70     arrive_at          timestampz not null,
71     status              varchar(20) not null default 'SCHEDULED'
72     | | | | check (status in ('SCHEDULED','DELAYED','CANCELLED','COMPLETED')),
73     gate               varchar(10),
74     created_at         timestampz not null default now(),
75     constraint chk_arrive_after_depart check (arrive_at > depart_at)
76 );
77
78 create index idx_schedules_flight on flight_schedules(flight_id);
79 create index idx_schedules_aircraft on flight_schedules(aircraft_id);
80
81 create table bookings (
82     booking_id         bigserial primary key,
83     user_id            bigint not null references users(user_id),
84     schedule_id        bigint not null references flight_schedules(schedule_id) on delete cascade,
85     booking_ref        varchar(12) not null unique,
86     status              varchar(20) not null default 'PENDING'
87     | | | | check (status in ('PENDING','CONFIRMED','CANCELLED')),
88     booked_at          timestampz not null default now()
89 );
90
91 create index idx_bookings_user on bookings(user_id);
92 create index idx_bookings_schedule on bookings(schedule_id);
93
94 create table tickets
95     ticket_id          bigserial primary key,
96     booking_id         bigint not null references bookings(booking_id) on delete cascade,
97     passenger_name     varchar(120) not null,
98     passport_no        varchar(30),
99     seat_no            varchar(5) not null,
100     ticket_price        numeric(10,2) not null check (ticket_price >= 0),
101     issued_at          timestampz not null default now()
102 );
103
104 create index idx_tickets_booking on tickets(booking_id);

```

```

106 create table payments (
107     payment_id         bigserial primary key,
108     booking_id         bigint not null references bookings(booking_id) on delete cascade,
109     amount             numeric(10,2) not null check (amount >= 0),
110     method             varchar(20) not null check (method in ('CARD','CASH','TRANSFER')),
111     paid_at            timestampz not null default now()
112 );
113
114 create index idx_payments_booking on payments(booking_id);
115

```

DML:

```
sql > dml_seed.sql
1 insert into roles(role_name, description) values
2 ('admin','Full access'),
3 ('employee','Manage flights/schedules'),
4 ('customer','Book flights'),
5 ('technical','Maintenance/support');
6
7 insert into users(username,email,password_hash,full_name,phone) values
8 ('admin','admin@flightdbms.com','demo_hash','Admin User','+90-000-000-0000'),
9 ('emp1','emp1@flightdbms.com','demo_hash','Employee One','+90-111-111-1111'),
10 ('tech1','tech1@flightdbms.com','demo_hash','Tech Staff','+90-222-222-2222'),
11 ('cust1','cust1@flightdbms.com','demo_hash','Customer One','+90-333-333-3333'),
12 ('cust2','cust2@flightdbms.com','demo_hash','Customer Two','+90-444-444-4444');
13
14 insert into user_roles(user_id, role_id)
15 select u.user_id, r.role_id from users u join roles r on r.role_name='admin' where u.username='admin';
16
17 insert into user_roles(user_id, role_id)
18 select u.user_id, r.role_id from users u join roles r on r.role_name='employee' where u.username='emp1';
19
20 insert into user_roles(user_id, role_id)
21 select u.user_id, r.role_id from users u join roles r on r.role_name='technical' where u.username='tech1';
22
23 insert into user_roles(user_id, role_id)
24 select u.user_id, r.role_id from users u join roles r on r.role_name='customer' where u.username in ('cust1','cust2');
25
26 insert into airports(iata_code,name,city,country) values
27 ('ECN','Ercan International Airport','Nicosia','Cyprus'),
28 ('IST','Istanbul Airport','Istanbul','Türkiye'),
29 ('SAW','Sabiha Gökçen Airport','Istanbul','Türkiye'),
30 ('LHR','Heathrow Airport','London','UK'),
31 ('DXB','Dubai International Airport','Dubai','UAE');
32
33 insert into aircraft(tail_number,model,manufacturer,seat_capacity) values
34 ('TC-AAA','A320','Airbus',180),
35 ('TC-BBB','B737-800','Boeing',189),
36 ('TC-CCC','A321','Airbus',220);
37
38 insert into flights(flight_no,from_airport_id,to_airport_id,base_price)
39 select 'CY101', a1.airport_id, a2.airport_id, 120
40 from airports a1, airports a2 where a1.iata_code='ECN' and a2.iata_code='IST';
41
42 insert into flights(flight_no,from_airport_id,to_airport_id,base_price)
43 select 'CY102', a1.airport_id, a2.airport_id, 95
44 from airports a1, airports a2 where a1.iata_code='IST' and a2.iata_code='ECN';
45
46 insert into flights(flight_no,from_airport_id,to_airport_id,base_price)
47 select 'TR201', a1.airport_id, a2.airport_id, 80
48 from airports a1, airports a2 where a1.iata_code='SAW' and a2.iata_code='IST';
49
50 insert into flights(flight_no,from_airport_id,to_airport_id,base_price)
51 select 'UK301', a1.airport_id, a2.airport_id, 250
52 from airports a1, airports a2 where a1.iata_code='IST' and a2.iata_code='LHR';
```



```

sql > dml_seed.sql
54 insert into flights(flight_no,from_airport_id,to_airport_id,base_price)
55 select 'AE401', a1.airport_id, a2.airport_id, 220
56 from airports a1, airports a2 where a1.iata_code='IST' and a2.iata_code='DXB';
57
58 insert into flight_schedules(flight_id,aircraft_id,depart_at,arrive_at,status,gate)
59 select f.flight_id, ac.aircraft_id, now() + interval '1 day', now() + interval '1 day 1 hour 20 min', 'SCHEDULED','A1'
60 from flights f join aircraft ac on ac.tail_number='TC-AAA' where f.flight_no='CY101';
61
62 insert into flight_schedules(flight_id,aircraft_id,depart_at,arrive_at,status,gate)
63 select f.flight_id, ac.aircraft_id, now() + interval '2 day', now() + interval '2 day 1 hour 15 min', 'SCHEDULED','B2'
64 from flights f join aircraft ac on ac.tail_number='TC-BBB' where f.flight_no='CY102';
65
66 insert into flight_schedules(flight_id,aircraft_id,depart_at,arrive_at,status,gate)
67 select f.flight_id, ac.aircraft_id, now() + interval '1 day 3 hour', now() + interval '1 day 3 hour 45 min', 'DELAYED','C3'
68 from flights f join aircraft ac on ac.tail_number='TC-CCC' where f.flight_no='TR201';
69
70 insert into flight_schedules(flight_id,aircraft_id,depart_at,arrive_at,status,gate)
71 select f.flight_id, ac.aircraft_id, now() + interval '3 day', now() + interval '3 day 4 hour', 'SCHEDULED','D4'
72 from flights f join aircraft ac on ac.tail_number='TC-AAA' where f.flight_no='UK301';
73
74 insert into flight_schedules(flight_id,aircraft_id,depart_at,arrive_at,status,gate)
75 select f.flight_id, ac.aircraft_id, now() + interval '4 day', now() + interval '4 day 3 hour 30 min', 'SCHEDULED','E5'
76 from flights f join aircraft ac on ac.tail_number='TC-BBB' where f.flight_no='AE401';
77
78 insert into bookings(user_id, schedule_id, booking_ref, status)
79 select u.user_id, s.schedule_id, '', 'PENDING'
80 from users u, flight_schedules s, flights f
81 where u.username='cust1' and s.flight_id=f.flight_id and f.flight_no='CY101'
82 limit 1;
83
84 insert into bookings(user_id, schedule_id, booking_ref, status)
85 select u.user_id, s.schedule_id, '', 'PENDING'
86 from users u, flight_schedules s, flights f
87 where u.username='cust2' and s.flight_id=f.flight_id and f.flight_no='CY101'
88 limit 1;
89
90 insert into bookings(user_id, schedule_id, booking_ref, status)
91 select u.user_id, s.schedule_id, '', 'PENDING'
92 from users u, flight_schedules s, flights f
93 where u.username='cust1' and s.flight_id=f.flight_id and f.flight_no='CY102'
94 limit 1;
95
96 insert into bookings(user_id, schedule_id, booking_ref, status)
97 select u.user_id, s.schedule_id, '', 'PENDING'
98 from users u, flight_schedules s, flights f
99 where u.username='cust2' and s.flight_id=f.flight_id and f.flight_no='UK301'
100 limit 1;
101
102 insert into bookings(user_id, schedule_id, booking_ref, status)
103 select u.user_id, s.schedule_id, '', 'PENDING'
104 from users u, flight_schedules s, flights f
105 where u.username='cust1' and s.flight_id=f.flight_id and f.flight_no='AE401'
106 limit 1;

```

```

108 insert into tickets(booking_id,passenger_name,passport_no,seat_no,ticket_price)
109 select b.booking_id,'Customer One','P12345','12A',140 from bookings b limit 1;
110
111 insert into tickets(booking_id,passenger_name,passport_no,seat_no,ticket_price)
112 select b.booking_id,'Customer Two','P23456','12B',140 from bookings b offset 1 limit 1;
113
114 insert into tickets(booking_id,passenger_name,passport_no,seat_no,ticket_price)
115 select b.booking_id,'Customer One','P12345','14C',110 from bookings b offset 2 limit 1;
116
117 insert into tickets(booking_id,passenger_name,passport_no,seat_no,ticket_price)
118 select b.booking_id,'Customer Two','P23456','2D',260 from bookings b offset 3 limit 1;
119
120 insert into tickets(booking_id,passenger_name,passport_no,seat_no,ticket_price)
121 select b.booking_id,'Customer One','P12345','8A',240 from bookings b offset 4 limit 1;
122
123 insert into payments(booking_id,amount,method)
124 select booking_id, ticket_price, 'CARD' from tickets limit 3;
125

```

Queries:

```

1 -- Q1: Upcoming schedules with full details (JOIN)
2 select
3   s.schedule_id,
4   f.flight_no,
5   ap1.iata_code as from_iata,
6   ap2.iata_code as to_iata,
7   s.depart_at,
8   s.arrive_at,
9   s.status,
10  a.tail_number,
11  a.model
12 from flight_schedules s
13 join flights f on f.flight_id = s.flight_id
14 join airports ap1 on ap1.airport_id = f.from_airport_id
15 join airports ap2 on ap2.airport_id = f.to_airport_id
16 join aircraft a on a.aircraft_id = s.aircraft_id
17 order by s.depart_at;

```

schedule_id	flight_no	from_iata	to_iata	depart_at	arrive_at	status	tail_number	model
1	CY101	ECN	IST	2025-12-22 14:22:52.54571+00	2025-12-22 15:42:52.54571+00	SCHEDULED	TC-AAA	A320
3	TR201	SAW	IST	2025-12-22 17:22:52.54571+00	2025-12-22 18:07:52.54571+00	CANCELLED	TC-CCC	A321
2	CY102	IST	ECN	2025-12-23 14:22:52.54571+00	2025-12-23 15:37:52.54571+00	SCHEDULED	TC-BBB	B737-800
6	TR201	SAW	IST	2025-12-24 13:56:00+00	2025-12-31 01:55:00+00	DELAYED	TC-BBB	B737-800
4	UK301	IST	LHR	2025-12-24 14:22:52.54571+00	2025-12-24 18:22:52.54571+00	SCHEDULED	TC-AAA	A320

```

1  -- Q2: Total bookings per flight_no (GROUP BY)
2  select
3      f.flight_no,
4      count(*) as total_bookings
5  from bookings b
6  join flight_schedules s on s.schedule_id = b.schedule_id
7  join flights f on f.flight_id = s.flight_id
8  group by f.flight_no
9  order by total_bookings desc;

```

Results Chart Export ▾

flight_no	total_booking	
CY101	2	
UK301	1	
CY102	1	
AE401	1	

```

1  -- Q3: Revenue per schedule (SUM via join)
2  select
3      s.schedule_id,
4      f.flight_no,
5      coalesce(sum(t.ticket_price),0) as revenue
6  from flight_schedules s
7  join flights f on f.flight_id = s.flight_id
8  left join bookings b on b.schedule_id = s.schedule_id
9  left join tickets t on t.booking_id = b.booking_id
10 group by s.schedule_id, f.flight_no
11 order by revenue desc;

```

Results Chart Export ▾

schedule_id	flight_no	revenue
4	UK301	260.00
2	CY102	250.00
5	AE401	240.00
1	CY101	140.00
3	TR201	0
6	TR201	0

```

1  -- Q4: Top routes by tickets sold (GROUP BY + ORDER)
2  select
3      ap1.iata_code as from_iata,
4      ap2.iata_code as to_iata,
5      count(t.ticket_id) as tickets_sold
6  from tickets t
7  join bookings b on b.booking_id = t.booking_id
8  join flight_schedules s on s.schedule_id = b.schedule_id
9  join flights f on f.flight_id = s.flight_id
10 join airports ap1 on ap1.airport_id = f.from_airport_id
11 join airports ap2 on ap2.airport_id = f.to_airport_id
12 group by ap1.iata_code, ap2.iata_code
13 order by tickets_sold desc
14 limit 5;

```

Results Chart Export ▾

from_iata	to_iata	tickets_sold	
IST	ECN	2	
IST	LHR	1	
IST	DXB	1	
ECN	IST	1	

```

1  -- Q5: Customers with most bookings
2  select
3      u.username,
4      u.full_name,
5      count(b.booking_id) as booking_count
6  from users u
7  left join bookings b on b.user_id = u.user_id
8  group by u.username, u.full_name
9  order by booking_count desc;

```

Results Chart Export ▾

username	full_name	booking_count	
cust1	Customer One	3	
cust2	Customer Two	2	
22209358	Abdu	0	
admin	Admin User	0	
tech1	Tech Staff	0	
emp1	Employee One	0	
abdu	Abdulraheem	0	
aa	aa	0	

```

1  -- Q6: Occupancy per schedule (tickets count) + capacity
2  select
3      s.schedule_id,
4      f.flight_no,
5      a.seat_capacity,
6      count(t.ticket_id) as sold_seats,
7      round( (count(t.ticket_id)::numeric / a.seat_capacity) * 100, 2 ) as occupancy_percent
8  from flight_schedules s
9  join flights f on f.flight_id = s.flight_id
10 join aircraft a on a.aircraft_id = s.aircraft_id
11 left join bookings b on b.schedule_id = s.schedule_id
12 left join tickets t on t.booking_id = b.booking_id
13 group by s.schedule_id, f.flight_no, a.seat_capacity
14 order by occupancy_percent desc;

```

Results Chart Export ▾

schedule_id	flight_no	seat_capacity	sold_seats	occupancy_per
2	CY102	189	2	1.06
4	UK301	180	1	0.56
1	CY101	180	1	0.56
5	AE401	189	1	0.53
3	TR201	220	0	0.00
6	TR201	189	0	0.00

```

1  -- Q7: Average ticket price by flight_no
2  select
3      f.flight_no,
4      round(avg(t.ticket_price),2) as avg_ticket_price
5  from tickets t
6  join bookings b on b.booking_id = t.booking_id
7  join flight_schedules s on s.schedule_id = b.schedule_id
8  join flights f on f.flight_id = s.flight_id
9  group by f.flight_no
10 order by avg_ticket_price desc;

```

Results Chart Export ▾

flight_no	avg_ticket_pr
UK301	260.00
AE401	240.00
CY101	140.00
CY102	125.00

Procedures:

Procedure created for confirming and canceling a booking

```
108 create or replace procedure confirm_booking_procedure(p_booking_id bigint)
109 language plpgsql
110 as $$
111 begin
112     update bookings
113     set status = 'CONFIRMED'
114     where booking_id = p_booking_id
115     and status <> 'CANCELLED';
116 end;
117 $$;
118 create or replace procedure cancel_booking_procedure(p_booking_id bigint)
119 language plpgsql
120 as $$
121 begin
122     update bookings
123     set status = 'CANCELLED'
124     where booking_id = p_booking_id;
125 end;
126 $$;
127
```

Results Chart Export ▾

Success. No rows returned

Table Before Calling the Confirm Procedure

booking_id	booking_ref	status
1	BK0000000001	CONFIRMED
2	BK0000000002	PENDING
3	BK0000000003	CONFIRMED
4	BK0000000004	PENDING
5	BK0000000005	PENDING

Table After Calling the Confirm Procedure

booking_id	booking_ref	status
1	BK0000000001	CONFIRMED
2	BK0000000002	PENDING
3	BK0000000003	CONFIRMED
4	BK0000000004	CONFIRMED
5	BK0000000005	PENDING

Table Before Cancel Procedure

booking_id	booking_ref	status
1	BK0000000001	CONFIRMED
2	BK0000000002	PENDING
3	BK0000000003	CONFIRMED
4	BK0000000004	CONFIRMED
5	BK0000000005	PENDING

1) Table After Cancel Procedure

booking_id	booking_ref	status
1	BK0000000001	CONFIRMED
2	BK0000000002	PENDING
3	BK0000000003	CONFIRMED
4	BK0000000004	CONFIRMED
5	BK0000000005	CANCELLED

Functions:

Function Created to Calculate Schedule Revenue

```
128 CREATE OR REPLACE FUNCTION calculate_schedule_revenue(p_schedule_id BIGINT)
129 RETURNS NUMERIC
130 LANGUAGE plpgsql
131 AS $$
132 DECLARE
133     total_revenue NUMERIC;
134 BEGIN
135     SELECT COALESCE(SUM(price), 0)
136     INTO total_revenue
137     FROM tickets t
138     JOIN bookings b ON b.booking_id = t.booking_id
139     WHERE b.schedule_id = p_schedule_id;
140
141     RETURN total_revenue;
142 END;
143 $$;
```

Results Chart Export ▾

Success. No rows returned

Function After Call + Output

```
1 select calculate_schedule_revenue(1);
2
```

Results Chart Export ▾

calculate_schedule_revenue
140.00

Trigger function (duplicate seat per schedule)

```
145 create or replace function prevent_duplicate_seat()
146 returns trigger
147 language plpgsql
148 as $$
149 declare
150     v_schedule_id bigint;
151 begin
152     select schedule_id into v_schedule_id
153     from bookings
154     where booking_id = new.booking_id;
155 if exists (
156     select 1
157     from tickets t
158     join bookings b on b.booking_id = t.booking_id
159     where b.schedule_id = v_schedule_id
160         and upper(t.seat_no) = upper(new.seat_no)
161 ) then
162     raise exception 'Seat % is already booked for this schedule', new.seat_no;
163 end if;
164 new.seat_no := upper(new.seat_no);
165 return new;
166 end; $$;
```

Results Chart Export ▾

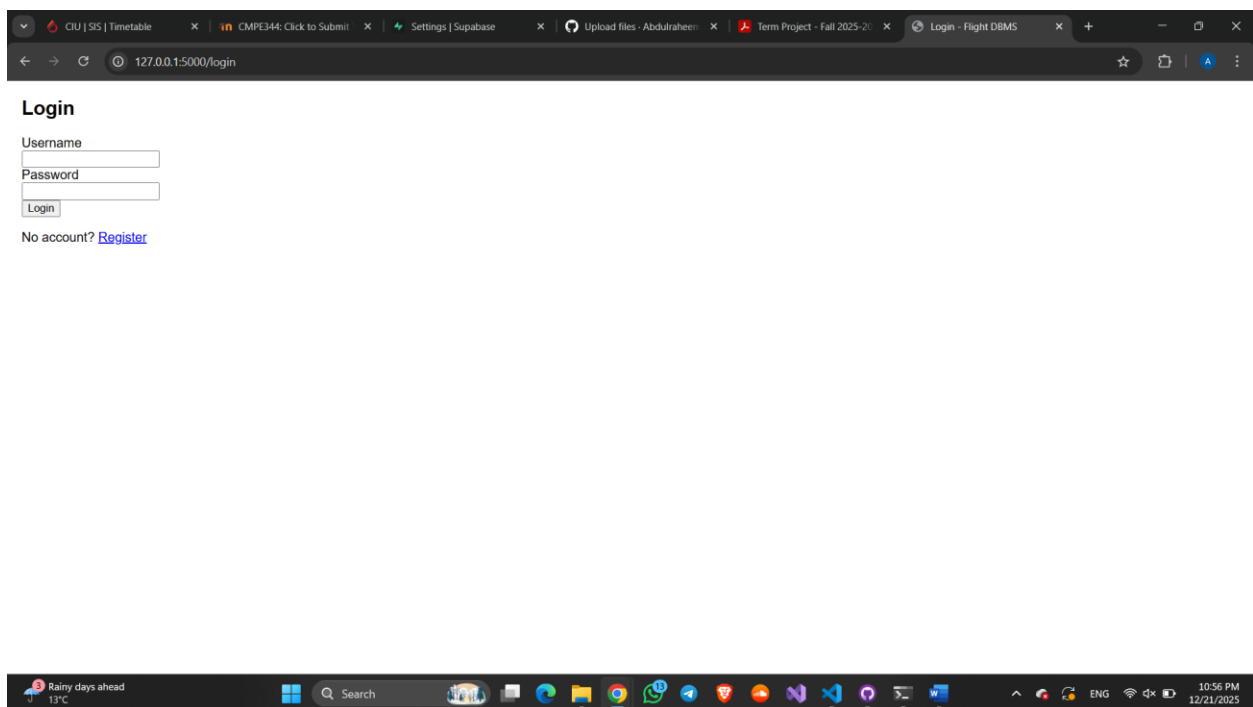
Success. No rows returned

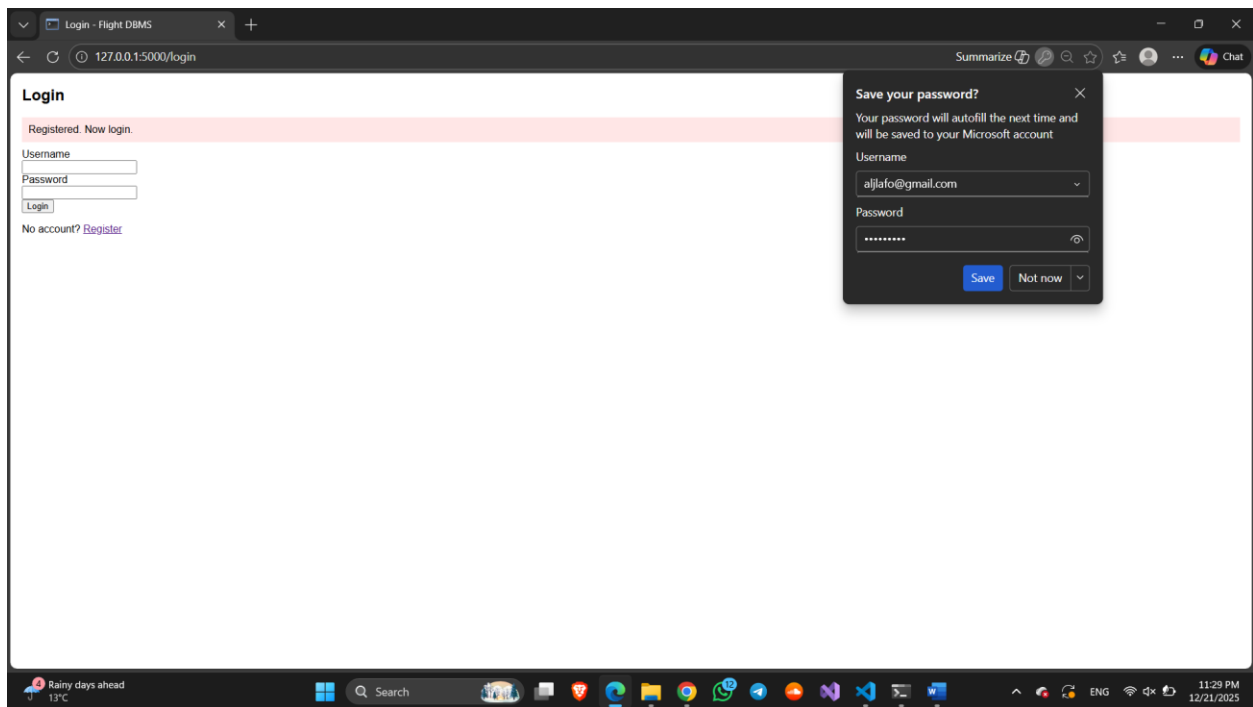
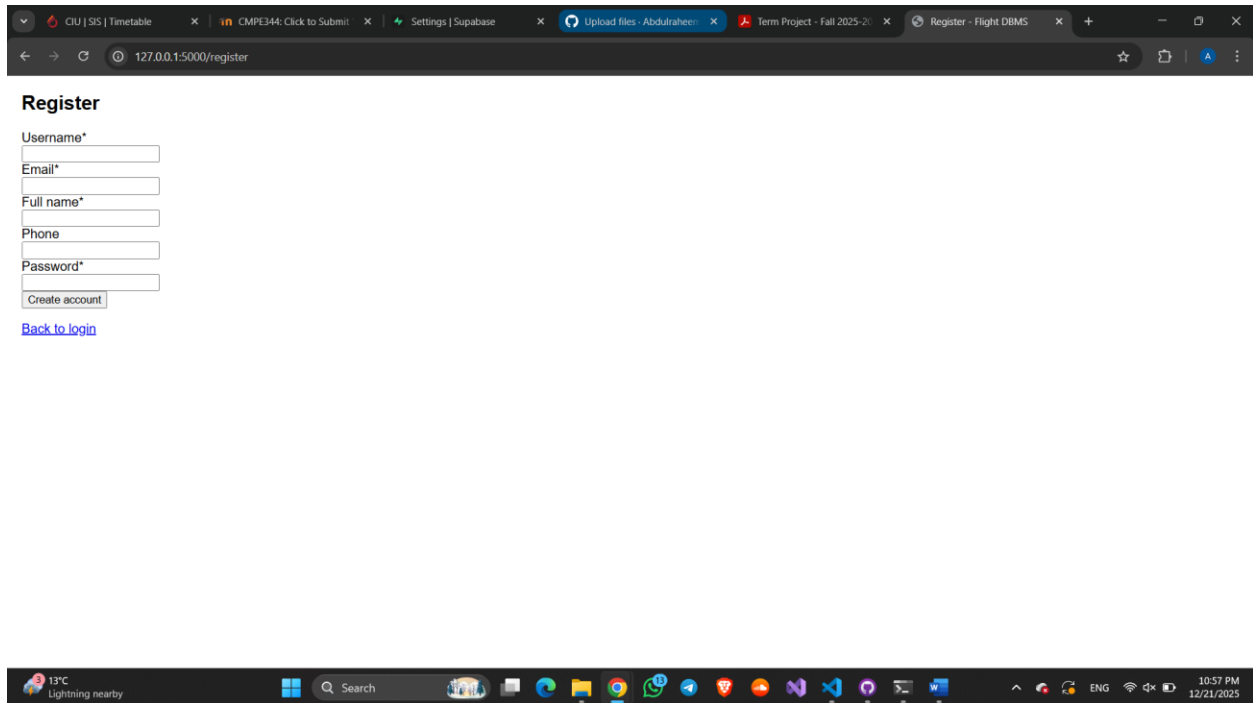
Software:

The GUI was developed using Flask and HTML templates.
The system includes three main management windows:

1. Flight Schedules Management
 - Add, update, and delete schedules
2. Bookings Management
 - Create and manage bookings
3. Tickets Management
 - Issue, update, and delete tickets

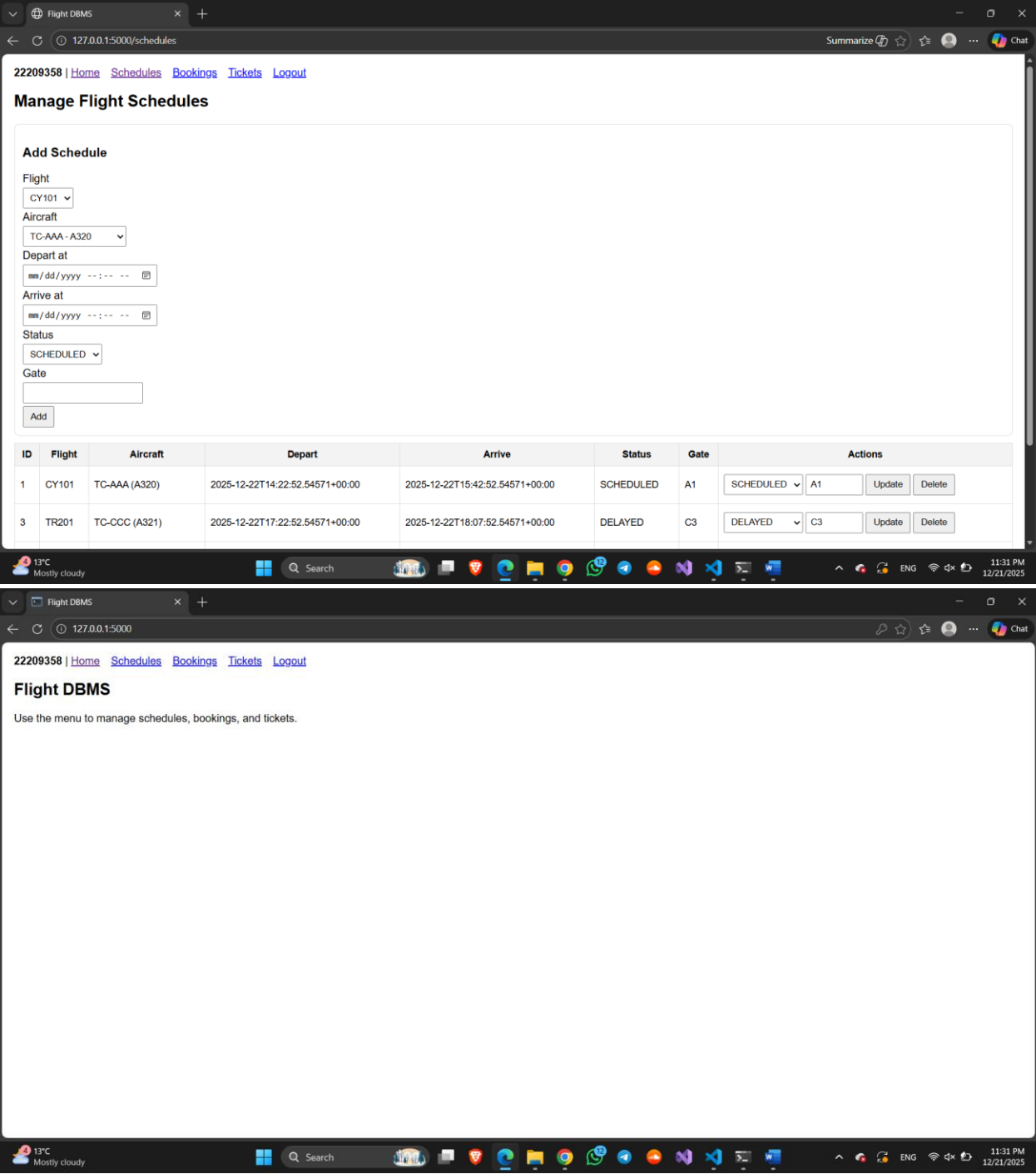
Each page supports full CRUD operations.

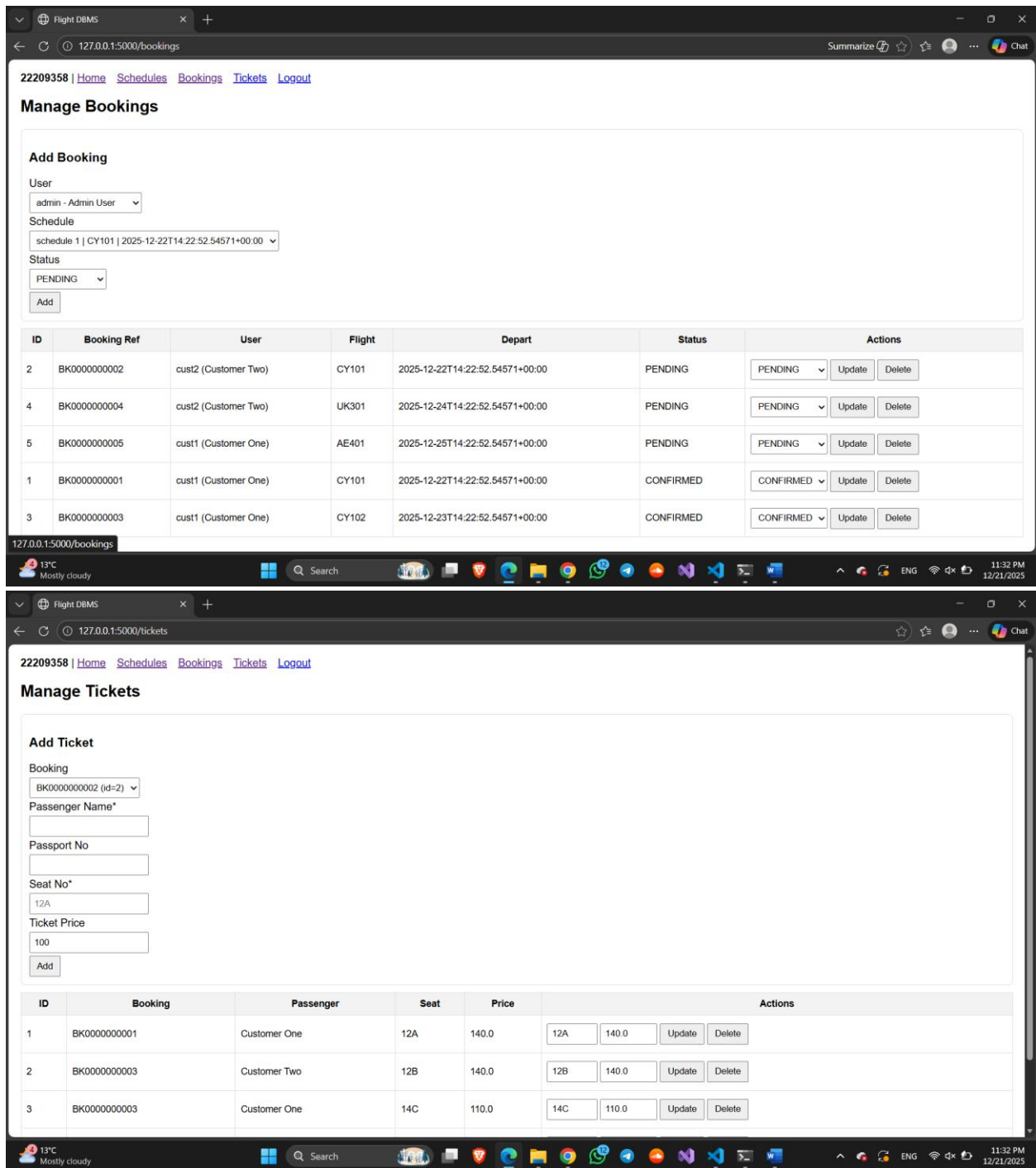




Nicosia, Turkish Republic of Northern Cyprus

2024/2025





Screenshots from the development environment (Flask server running, SQL Editor execution) demonstrate successful system operation.

Nicosia, Turkish Republic of Northern Cyprus

2024/2025

Slice of the Supabase Python Client

```
@app.post("/schedules/update")
@login_required
def schedules_update():
    schedule_id = int(request.form.get("schedule_id"))
    status = request.form.get("status")
    gate = request.form.get("gate", "").strip() or None

    try:
        supabase.table("flight_schedules").update({
            "status": status,
            "gate": gate
        }).eq("schedule_id", schedule_id).execute()
        flash("Schedule updated.", "ok")
    except Exception as e:
        flash(f"Update failed: {e}", "error")

    return redirect(url_for("schedules"))

@app.post("/schedules/delete")
@login_required
def schedules_delete():
    schedule_id = int(request.form.get("schedule_id"))
    try:
        supabase.table("flight_schedules").delete().eq("schedule_id", schedule_id).execute()
        flash("Schedule deleted.", "ok")
    except Exception as e:
        flash(f"Delete failed: {e}", "error")
    return redirect(url_for("schedules"))

@app.get("/bookings")
@login_required
def bookings():
    bookings_resp = supabase.table("bookings").select("*").order("booked_at", desc=True).execute()
    users_resp = supabase.table("users").select("user_id, username, full_name").execute()
    schedules_resp = supabase.table("flight_schedules").select("schedule_id, flight_id, depart_at, status").execute()
    flights_resp = supabase.table("flights").select("flight_id, flight_no").execute()

    users = {u["user_id"]: u for u in users_resp.data}
    flights = {f["flight_id"]: f for f in flights_resp.data}

    sched_rows = []
    for s in schedules_resp.data:
        sched_rows.append({
            **s,
            "flight_no": flights.get(s["flight_id"], {}).get("flight_no", "")
        })
    schedules_map = {s["schedule_id"]: s for s in sched_rows}

    rows = []
    for b in bookings_resp.data:
        u = users.get(b["user_id"], {})
        s = schedules_map.get(b["schedule_id"], {})
        rows.append({
```


GitHub:

All project files are published in a public GitHub repository to support collaboration and transparency.

Repository link:

 <https://github.com/Abdulraheem-Bawazir/CMPE-344-Project-FightDBMS>

Repository Contents

- sql/ → DDL, DML, queries, functions & triggers
- app/ → Flask application code
- templates/ → HTML templates
- ERD/ → ER diagram
- screenshots/ → Application and database screenshots
- README.md → Setup and usage instructions

Version control was handled using Git, ensuring proper documentation and traceability of changes.